Course/Section EET 316L *Name:* Bryce Poonai *Team#* N/A

Lab Cover Sheet

Farmingdale State College

***S**tate **U**niversity of **N**ew **Y**ork*

Electrical Engineering Technology

Experiment No. 13&14

Title: Vending Machine Project

Course/Section EET 316L *Name:* Bryce Poonai *Team#* N/A

Objective:

The objective of this Lab Final Project is to build a vending machine using a finite state machine. The state diagram is given in the lab procedure. This project is to be implemented using VHDL by using the ISE Design Suite. Then there are a couple ways to test and demonstrate this project. The first way is to demo the project on the Spartan boards. After that, test bench code can be created to simulate inputs. Then, students can use the Analog Discovery boards and software to further test the vending machine. VHDL code, UCF code, and Output Waveforms are to be recorded.

Equipment:

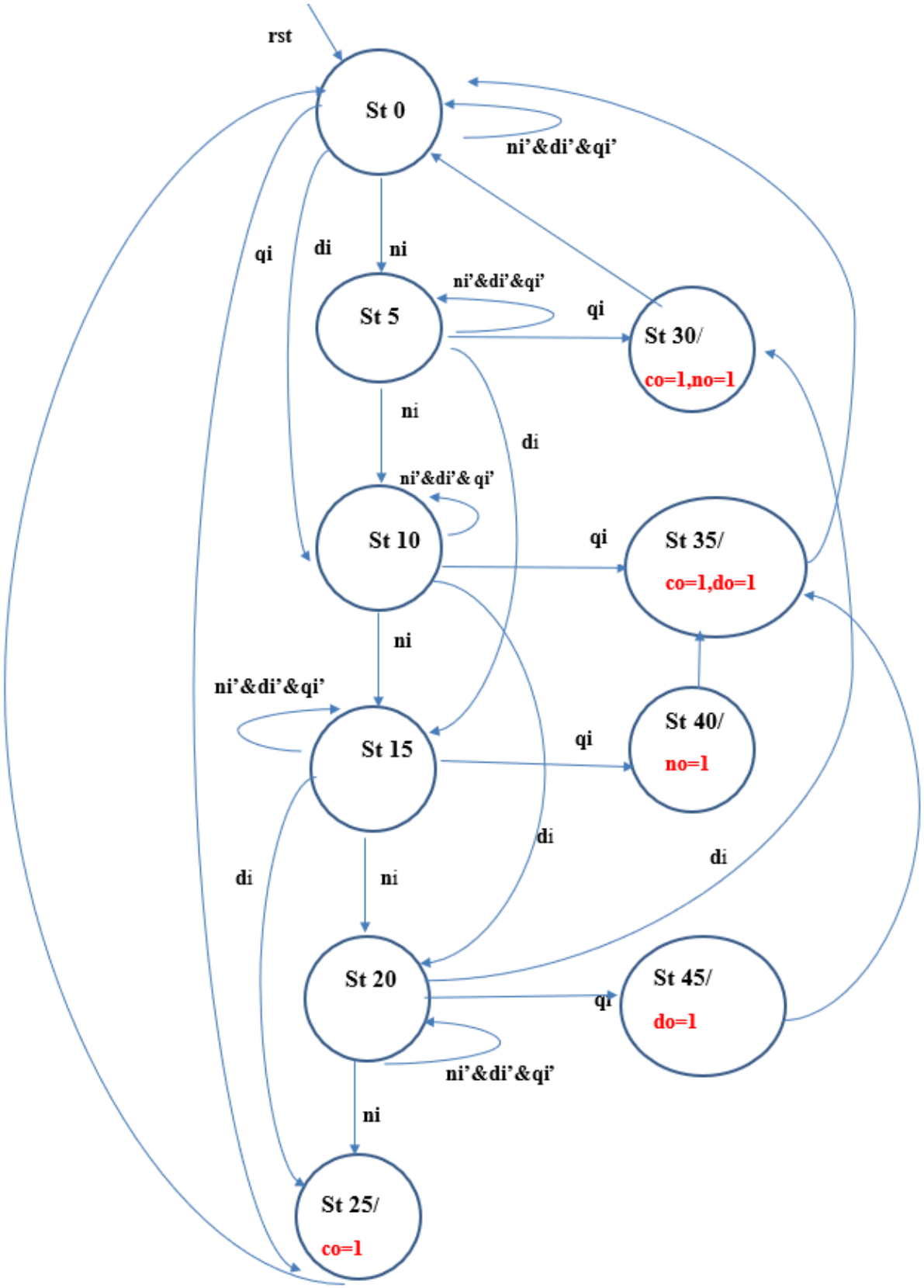
Item	Description
1.	Digilent Nexys 3 Board with Spartan 6 XC6LX16-CS324 FPGA
2.	ISE Design Suite 14.1
3.	Digilent Adept
4.	Analog Discovery Boards
5.	Analog Discovery Waveforms

Course/Section EET 316L *Name:* Bryce Poonai *Team#* N/A

Data Tables:

Vending Machine Project:

State Diagram:



Course/Section EET 316L *Name:* Bryce Poonai *Team#* N/A

ISE Design Suite Section:

VHDL Code:

```
----------------------------------------------------------------------------------
-- Company:
-- Engineer:
--
-- Create Date:    09:47:19 05/04/2022
-- Design Name:
-- Module Name:    VendingMachine - FSM
-- Project Name:
-- Target Devices:
-- Tool versions:
-- Description:
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
USE ieee.std_logic_unsigned.all;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx primitives in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity VendingMachine is
    Port ( nickel_in : in  STD_LOGIC;
           dime_in : in  STD_LOGIC;
           quarter_in : in  STD_LOGIC;
           clk : in  STD_LOGIC;
           reset : in  STD_LOGIC;
           clkout : out STD_LOGIC;
           candy_out : out  STD_LOGIC;
           nickel_out : out  STD_LOGIC;
           dime_out : out  STD_LOGIC);
end VendingMachine;

architecture FSM of VendingMachine is
SIGNAL Count: STD_LOGIC_VECTOR (26 downto 0);
SIGNAL clkoutFSM: STD_LOGIC;
type state_type is (st0, st5, st10, st15, st20, st25, st30, st35, st40, st45);
signal current_state, next_state: state_type;
begin
```

```
52  --slow clock
53  PROCESS (clk, reset)
54  BEGIN
55  IF reset='1' then
56  Count<="00000000000000000000000000000000";
57  ELSIF clk'EVENT AND clk='1' THEN
58  Count<=Count+1;
59  END IF;
60  END PROCESS;
61  clkout<=Count(1);
62  clkoutFSM <= Count(1);
63
64
65  --next state decoder
66  process (current_state, nickel_in, dime_in, quarter_in)
67  begin
68  case current_state is
69  when st0 =>
70  if nickel_in = '1' then next_state <= st5;
71  elsif dime_in = '1' then next_state <= st10;
72  elsif quarter_in = '1' then next_state <= st25;
73  else next_state <= st0;
74  end if;
75  when st5 =>
76  if nickel_in = '1' then next_state <= st10;
77  elsif dime_in = '1' then next_state <= st15;
78  elsif quarter_in = '1' then next_state <= st30;
79  else next_state <= st5;
80  end if;
81  when st10 =>
82  if nickel_in = '1' then next_state <= st15;
83  elsif dime_in = '1' then next_state <= st20;
84  elsif quarter_in = '1' then next_state <= st35;
85  else next_state <= st10;
86  end if;
87  when st15 =>
88  if nickel_in = '1' then next_state <= st20;
89  elsif dime_in = '1' then next_state <= st25;
90  elsif quarter_in = '1' then next_state <= st40;
91  else next_state <= st15;
92  end if;
93  when st20 =>
94  if nickel_in = '1' then next_state <= st25;
95  elsif dime_in = '1' then next_state <= st30;
96  elsif quarter_in = '1' then next_state <= st45;
97  else next_state <= st20;
98  end if;
99  when st25 =>
100 next_state <= st0;
101 when st30 =>
102 next_state <= st0;
103 when st35 =>
104 next_state <= st0;
105 when st40 =>
106 next_state <= st35;
```

```
107  when st45 =>
108  next_state <= st35;
109  when OTHERS =>
110  next_state <= st0;
111  end case;
112  end process;
113
114
115  --state memory
116  process (clkoutFSM, reset)
117  begin
118  if reset = '1' then
119  current_state <= st0;
120  elsif clkoutFSM'event and clkoutFSM = '1' then
121  current_state <= next_state;
122  end if;
123  end process;
124
125
126  --output decoder
127  process (current_state)
128  begin
129  case current_state is
130  when st25 =>
131  candy_out <= '1';
132  nickel_out <= '0';
133  dime_out <= '0';
134  when st30 =>
135  candy_out <= '1';
136  nickel_out <= '1';
137  dime_out <= '0';
138  when st35 =>
139  candy_out <= '1';
140  dime_out <= '1';
141  nickel_out <= '0';
142  when st40 =>
143  nickel_out <= '1';
144  candy_out <= '0';
145  dime_out <= '0';
146  when st45 =>
147  dime_out <= '1';
148  candy_out <= '0';
149  nickel_out <= '0';
150  when OTHERS =>
151  candy_out <= '0';
152  nickel_out <= '0';
153  dime_out <= '0';
154  end case;
155  end process;
156
157
158  end FSM;
159
```

Course/Section EET 316L *Name:* Bryce Poonai *Team#* N/A

UCF File:

```
#Inputs
NET"nickel_in"LOC=T10;
NET"dime_in"LOC=T9;
NET"quarter_in"LOC=V9;
NET"reset"LOC=M8;
NET"clk"LOC=V10;
#Outputs
NET"nickel_out"LOC=U16;
NET"dime_out"LOC=V16;
NET"candy_out"LOC=U15;
NET"clkout"LOC=V15;
```

Test Bench Code:

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--USE ieee.numeric_std.ALL;

ENTITY FSMTB IS
END FSMTB;

ARCHITECTURE behavior OF FSMTB IS

    -- Component Declaration for the Unit Under Test (UUT)

    COMPONENT VendingMachine
    PORT(
         nickel_in : IN  std_logic;
         dime_in : IN  std_logic;
         quarter_in : IN  std_logic;
         clk : IN  std_logic;
         reset : IN  std_logic;
         clkout : OUT  std_logic;
         candy_out : OUT  std_logic;
         nickel_out : OUT  std_logic;
         dime_out : OUT  std_logic
        );
    END COMPONENT;


   --Inputs
   signal nickel_in : std_logic := '0';
   signal dime_in : std_logic := '0';
   signal quarter_in : std_logic := '0';
   signal clk : std_logic := '0';
   signal reset : std_logic := '1';

   --Outputs
   signal clkout : std_logic;
   signal candy_out : std_logic;
   signal nickel_out : std_logic;
   signal dime_out : std_logic;

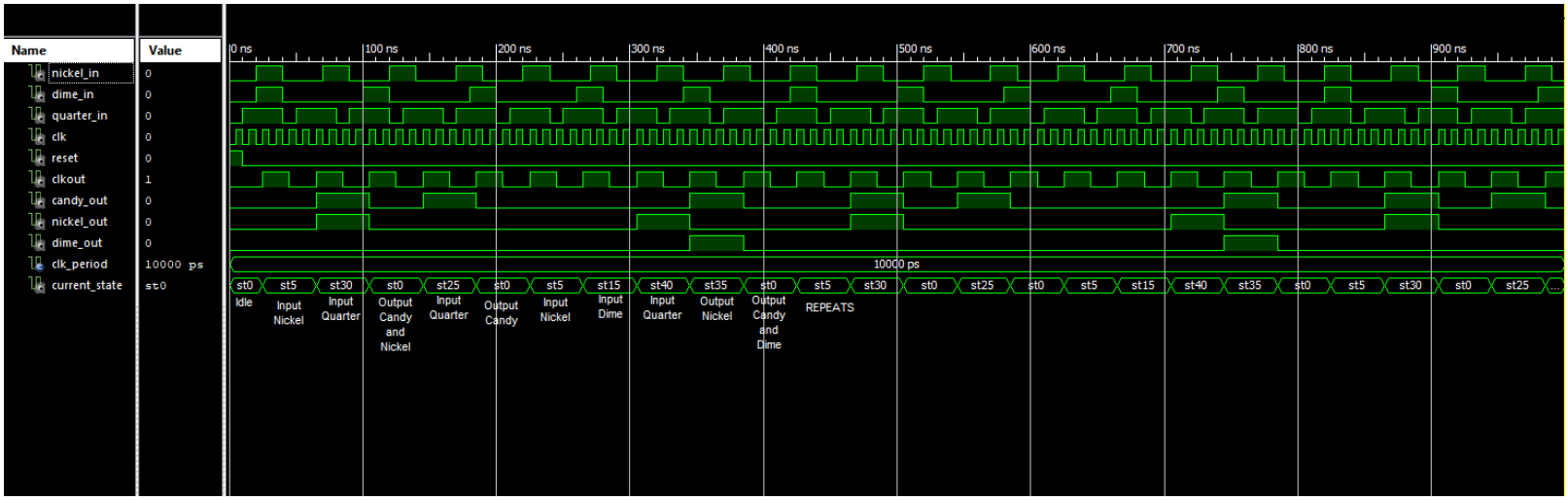
   -- Clock period definitions
   constant clk_period : time := 10 ns;

BEGIN
```

```
74
75      -- Instantiate the Unit Under Test (UUT)
76      uut: VendingMachine PORT MAP (
77             nickel_in => nickel_in,
78             dime_in => dime_in,
79             quarter_in => quarter_in,
80             clk => clk,
81             reset => reset,
82             clkout => clkout,
83             candy_out => candy_out,
84             nickel_out => nickel_out,
85             dime_out => dime_out
86           );
87
88      -- Clock process definitions
89      clk_process :process
90      begin
91         clk <= '0';
92         wait for clk_period/2;
93         clk <= '1';
94         wait for clk_period/2;
95      end process;
96
97      reset_process :process
98      begin
99         wait for clk_period;
100        reset <= '0';
101        wait for clk_period * 12;
102     end process;
103
104     nickel_process :process
105     begin
106        wait for clk_period * 2;
107        nickel_in <= '1';
108        wait for clk_period * 2;
109        nickel_in <= '0';
110        wait for clk_period;
111     end process;
112
113     dime_process :process
114     begin
115        wait for clk_period * 2;
116        dime_in <= '1';
117        wait for clk_period * 2;
118        dime_in <= '0';
119        wait for clk_period * 4;
120     end process;
121
122     quarter_process :process
123     begin
124        wait for clk_period;
125        quarter_in <= '1';
126        wait for clk_period * 3;
127        quarter_in <= '0';
128     end process;
129
130
131
132
133
134
135  END;
```

Course/Section EET 316L *Name:* Bryce Poonai *Team#* N/A

Labeled Test Bench Waveforms:



Analog Discovery Section:

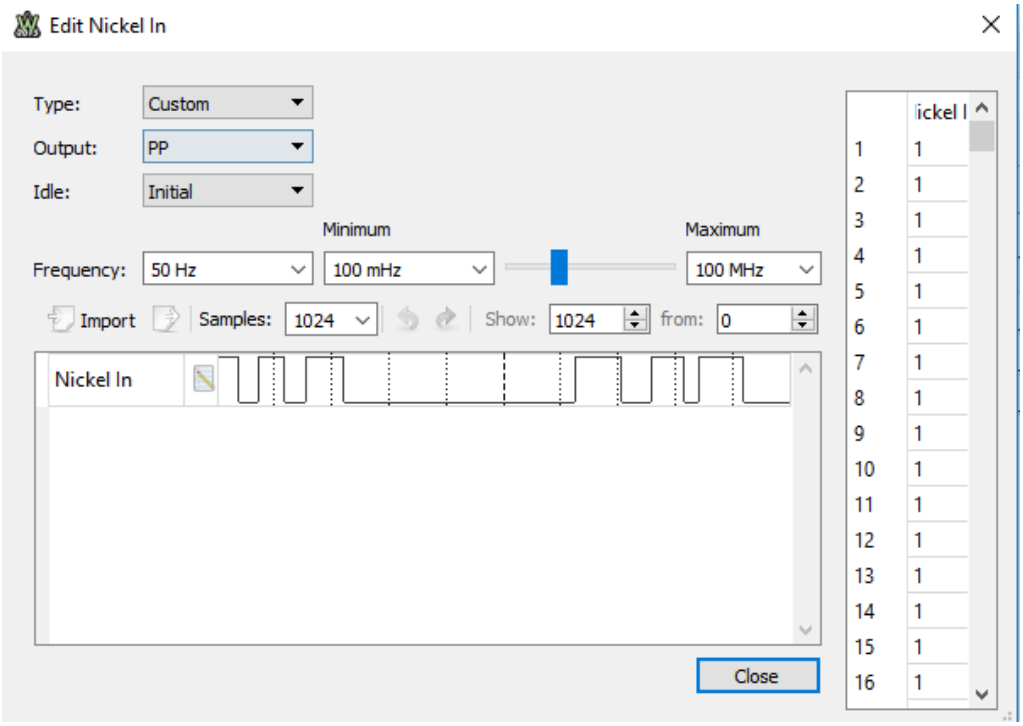
UCF File:

```
#Inputs
NET"nickel_in"LOC=M10;
NET"dime_in"LOC=N9;
NET"quarter_in"LOC=U11;
NET"reset"LOC=V11;
NET"clk"LOC=V10;
#Outputs
NET"nickel_out"LOC=V12;
NET"dime_out"LOC=N10;
NET"candy_out"LOC=P11;
NET"clkout"LOC=T12;
```

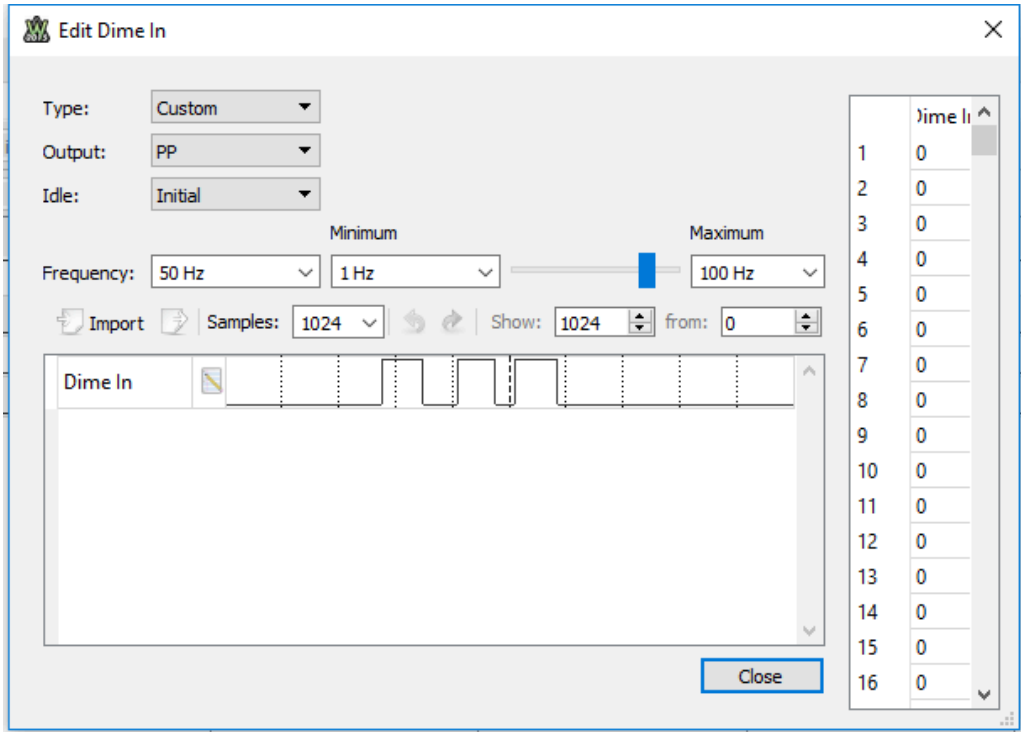
Course/Section EET 316L *Name:* Bryce Poonai *Team#* N/A

Custom Input Waveform Settings for Accessing Every State:

Nickel In:

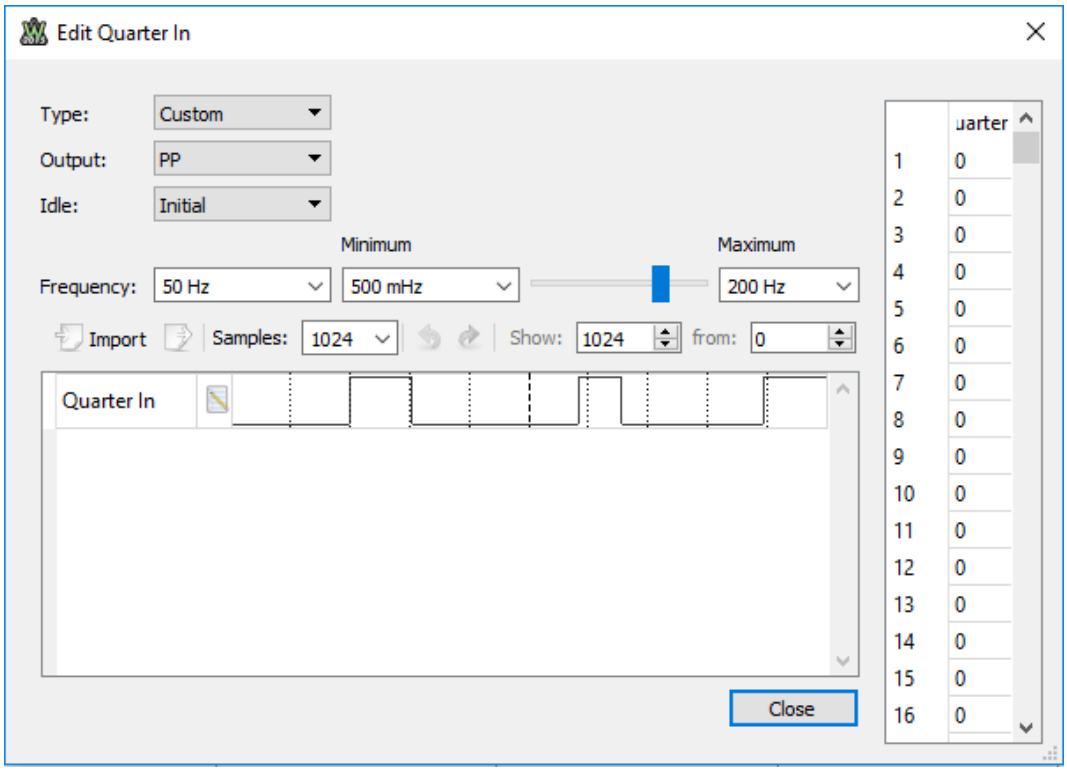


Dime In:

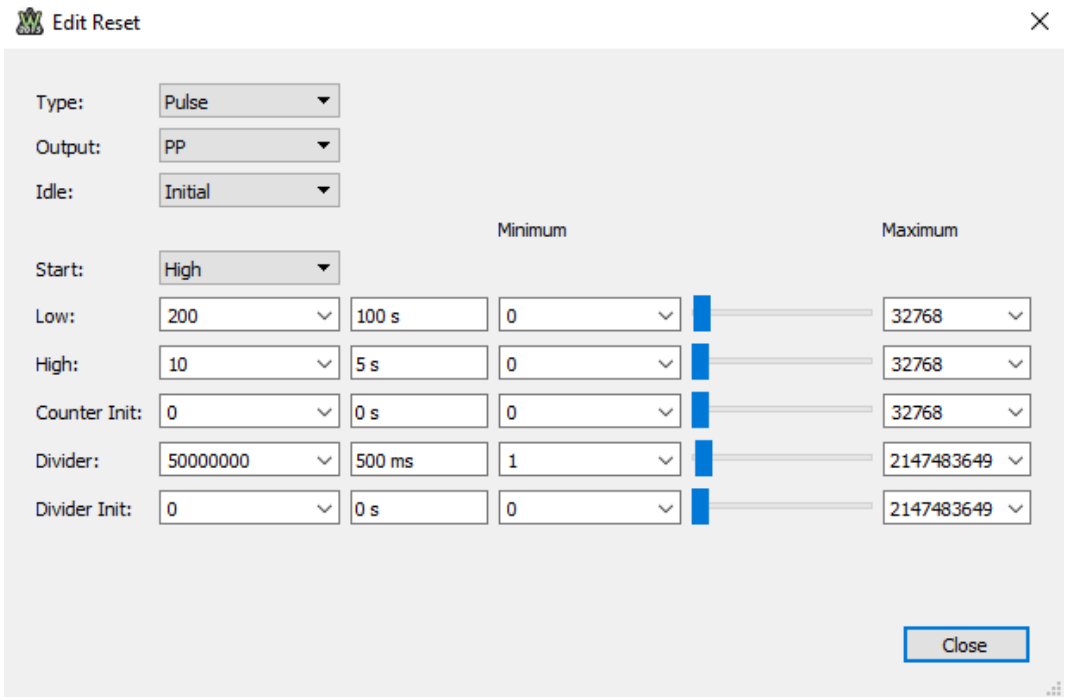


Course/Section EET 316L *Name:* Bryce Poonai *Team#* N/A

Quarter In:



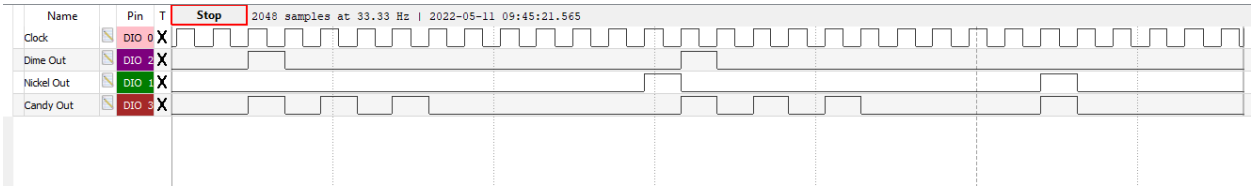
Reset Pulse:



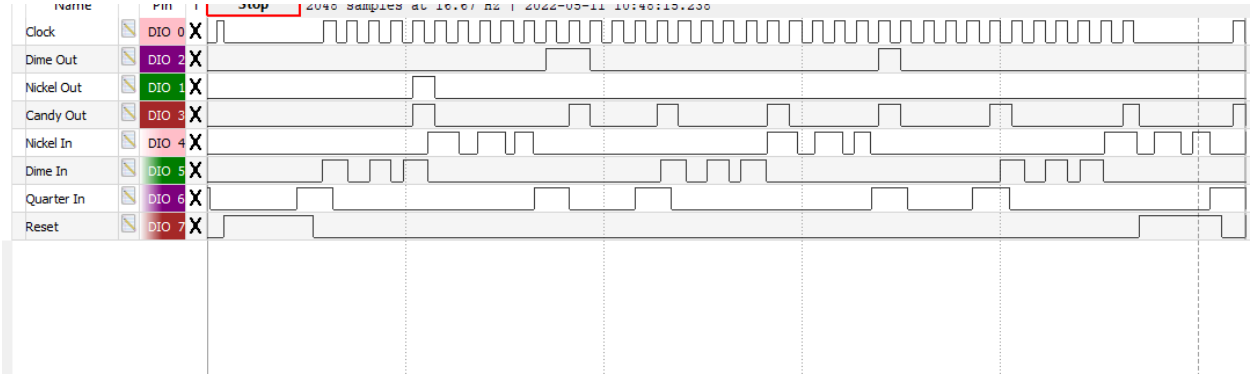
Course/Section EET 316L *Name:* Bryce Poonai *Team#* N/A

Analog Discovery Waveforms:

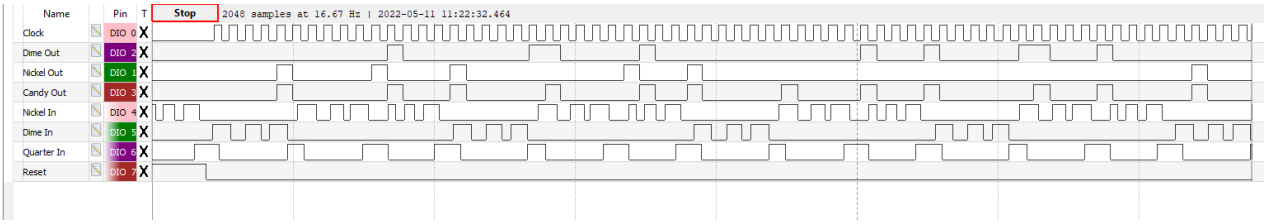
Using Input Switches:



Using Input Waveforms:



Every State Using Input Waveforms:



Conclusion:

The vending machine project demonstrates the fundamentals of what the finite state machine is. In this lab project, VHDL code was built to satisfy all the states with their respective outputs from the state diagram. Nickels, quarters, and dimes are the inputs that control what state we are in. The vending machine project was demonstrated four different ways. The first way was through the boards. This is where the slow clock was used to make it possible to operate the vending machine on the boards. Then after the boards, functional verification was performed by building test bench code and reviewing output waveform results. This is where I actually debugged an error that I wasn't able to see when testing the project on the boards. When in states like 40 or 45, a nickel or dime would output out of the vending machine. But then after, you would jump to another state that outputs the candy bar. The problem was that nickel_out or dime_out

would stay on even after they jumped to a state with a candy bar. I fixed that issue by accounting for every output within the output decoder. So, functional verification helped debug my code. As for the Analog Discovery testing, the first thing I did was simulate the outputs while using the board switches. Then after that, I simulated the inputs like we did with functional verification. I tried to test all states by using the custom waveform tools. They allowed you to create a custom waveform that can be clocked at a wide frequency range to choose from. All the custom waveform settings' screenshots should show the options I chose. The Analog Discovery platform is a great choice to perform functional verification on this project. I really like how much customization you have for simulating inputs. I feel like with more time, I can maximize all the options the Analog Discovery Waveforms software has to offer. Finally, there weren't any issues while working on this project.